

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: «Коммивояжёр»
Вариант 7

Студент гр. 4343

Стариков Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные: n — количество городов ($5 \leq n \leq 15$). Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные: Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город. Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0. Если такого пути не существует, вывести "no path".

Описание алгоритмов и оценка их сложности во времени и памяти

1.1. Точный метод: рекурсивное динамическое программирование

Задача коммивояжёра (TSP) формулируется так: найти гамильтонов цикл минимальной стоимости, проходящий через все заданные города ровно один раз и возвращающийся в исходный город. Для точного решения используется метод динамического программирования. Состояние задаётся парой $(mask, last)$, где $mask$ – битовая маска уже посещённых городов (бит $i = 1$, если город i посещён), а $last$ – город, в котором мы находимся в данный момент. Значение $dp[mask][last]$ – минимальная стоимость пути, который начинается в городе 0, проходит ровно через все города из $mask$ и заканчивается в $last$, а затем посещает все оставшиеся города и возвращается в 0. Базовый случай: $mask$ содержит все города ($mask == (1 \ll n) - 1$). Тогда из $last$ нужно вернуться в 0: если ребро существует, $dp[mask][last] = graph[last][0]$, иначе INF .

Вычисления выполняются рекурсивно с мемоизацией (результаты сохраняются в таблице dp). Дополнительно заполняется таблица $nextMove[mask][last]$, в которой для каждого состояния хранится следующий город, обеспечивающий оптимальное продолжение. Это позволяет восстановить сам маршрут после нахождения минимальной стоимости.

Оценка сложности:

Временная сложность: количество состояний равно $2^n * n$. Для каждого состояния перебираются все возможные следующие города (до n). Таким образом, общее число операций составляет $O(n^2 * 2^n)$.

Память: таблицы dp и $nextMove$ имеют размер $(2^n) * n$. Каждая ячейка хранит целое число (стоимость или следующий город). Также используется стек рекурсии глубиной не более n . Итоговая память – $O(n * 2^n)$.

1.2. Приближённый метод АЛШ-1 (жадный алгоритм ближайшего соседа)

Идея алгоритма АЛШ-1 – эвристический алгоритм, который строит маршрут, начиная с заданной стартовой вершины (город 0). На каждом шаге из текущего города выбирается ближайший ещё не посещённый город (с минимальным расстоянием, но не равным 0). После посещения всех городов выполняется попытка вернуться в начальный город 0. Если на каком-то шаге нет доступного непосещённого города или в конце отсутствует ребро обратно в 0, маршрут считается невозможным. Алгоритм не гарантирует оптимальности, но работает быстро и часто даёт приемлемое решение.

Оценка сложности:

Временная сложность: на каждом из n шагов необходимо просмотреть все непосещённые города (до n). Итого $O(n^2)$.

Память: требуется хранить массив посещённых городов ($O(n)$), текущий маршрут ($O(n)$) и исходную матрицу ($O(n^2)$). Общая память – $O(n^2)$.

Описание функций для точного алгоритма (рекурсивное ДП)

В реализации точного метода используются следующие функции:

```
int tsp_dp(const std::vector<std::vector<int>>& graph, std::vector<int>& path)
```

Назначение: вычисляет минимальную стоимость гамильтонова цикла, начинающегося и заканчивающегося в городе 0, а также восстанавливает оптимальный маршрут.

Параметры:

`graph` – матрица расстояний размером $n \times n$. Значение `graph[i][j]` – расстояние от города i до города j . Если $i \neq j$ и `graph[i][j] == 0`, то прямого пути нет.

`path` – выходной параметр: вектор, в который будет записан оптимальный маршрут (последовательность городов, начинающаяся и заканчивающаяся городом 0). В случае отсутствия пути вектор очищается.

Возвращаемое значение: минимальная стоимость маршрута, либо INF ($1e9$), если маршрут не существует.

Логика работы:

1. Получает количество городов $n = \text{graph.size}()$.
2. Создает таблицы `dp` (для мемоизации) и `nextMove` (для восстановления пути) размера $(1 \leq n) \times n$, инициализирует их значением -1.
3. Запускает рекурсивную вспомогательную функцию `tsp_dp_rec` с начальной маской $1 \ll 0$ (только город 0) и последним городом 0.
4. Если результат `bestCost` не равен INF, восстанавливает маршрут, проходя по ссылкам `nextMove` от начального состояния до полной маски, после чего добавляет возврат в 0.
5. Возвращает `bestCost` и заполняет `path`.

```
static int tsp_dp_rec(int mask, int last, const std::vector<std::vector<int>>&
graph, std::vector<std::vector<int>>& dp, std::vector<std::vector<int>>&
nextMove, int n)
```

Назначение: рекурсивно вычисляет минимальную стоимость завершения маршрута из состояния (mask, last), где mask – битовая маска уже посещённых городов, last – текущий город.

Параметры:

mask – битовая маска посещённых городов. last – индекс последнего посещённого города (входит в mask).

graph – матрица расстояний. dp – таблица для мемоизации (передаётся по ссылке).

nextMove – таблица для сохранения следующего города (передаётся по ссылке).

n – количество городов.

Возвращаемое значение: минимальная стоимость завершения маршрута (включая возврат в 0) из данного состояния.

Логика работы:

1. Проверяет, вычислено ли уже состояние ($dp[mask][last] \neq -1$). Если да – возвращает сохранённое значение.
2. Если $mask == (1 \ll n) - 1$ (все города посещены): Если есть ребро из last в 0, сохраняет $nextMove[mask][last] = 0$, запоминает $dp[mask][last] = graph[last][0]$ и возвращает эту стоимость. Иначе возвращает INF.
3. Иначе перебирает все города j от 0 до n-1: Если j уже в маске или нет прямого пути $last \rightarrow j$ – пропускает. Вычисляет $newMask = mask | (1 \ll j)$. Рекурсивно вызывает $tsp_dp_rec(newMask, j, \dots)$ и получает subCost. Полная стоимость перехода через j равна $graph[last][j] + subCost$. Выбирает минимальную стоимость и запоминает лучший следующий город bestNext.
4. Сохраняет $nextMove[mask][last] = bestNext$ и $dp[mask][last] = best$.

5. Возвращает best.

`std::string maskToBinary(int mask, int n)` (вспомогательная). Преобразует целочисленную маску в двоичную строку длины `n` (для наглядного вывода).

Описание функций для приближённого алгоритма АЛШ-1

`int tsp_alsh1(const std::vector<std::vector<int>>& graph, std::vector<int>& path)`

Назначение: строит гамильтонов цикл с помощью жадного алгоритма ближайшего соседа, начиная из города 0.

Возвращает стоимость построенного маршрута (или INF, если маршрут невозможен).

Параметры:

`graph` – матрица расстояний.

`path` – выходной параметр: вектор с маршрутом (начинается и заканчивается в 0) или пустой, если пути нет.

Возвращаемое значение: стоимость найденного маршрута, либо INF.

Логика работы:

1. Определяет `n = graph.size()`.
2. Инициализирует массив `visited` (все `false`), текущий город `current = 0`, `totalCost = 0`, `path = {0}`, `visited[0] = true`.
3. Для каждого шага `step` от 1 до `n-1`: Перебирает все непосещённые города `j` и находит среди них ближайший к `current` (минимальное `graph[current][j] > 0`). Если подходящий город не найден (`nearest == -1`), устанавливает флаг `possible = false` и выходит из цикла. Добавляет найденный город в путь, обновляет `totalCost`, переключает `current` и отмечает город посещённым.
4. После цикла, если `possible == true`, проверяет наличие ребра из последнего `current` в 0: если есть – добавляет его стоимость, добавляет 0 в путь и возвращает `totalCost`. Иначе – маршрут невозможен, очищает `path` и возвращает INF.
5. Если маршрут невозможен на любом из этапов, очищает `path` и возвращает INF.

Тестирование

Для проверки корректности и эффективности разработанных алгоритмов было проведено автоматизированное тестирование с использованием фреймворка Google Test. Тесты покрывают как образцы из условия задачи, так и специальные случаи, включая отсутствие пути и ситуации, где жадный алгоритм заведомо неоптимален.

Набор тестов для точного метода (DP):

1. Sample Input 1 – матрица 5×5 из условия. Ожидаемая минимальная стоимость = 78, маршрут 0 4 2 3 1 0.
2. Отсутствие пути – граф, в котором нельзя построить гамильтонов цикл (например, три города, соединённые линейно). Ожидается INF.
3. Треугольник – три города с полной симметричной матрицей. Оптимальная стоимость = 17, путь 0 1 2 0.
4. Полный граф из 4 городов – расстояния заданы так, что оптимальный цикл очевиден. Проверяется корректность восстановления пути.
5. Два города – тривиальный случай: стоимость = сумма двух ребер, маршрут 0 1 0.

Набор тестов для приближённого метода АЛШ-1:

1. Sample Input 1 – ожидается жадный маршрут 0 1 2 4 3 0 стоимостью 106 ($1+15+28+39+23$).
2. Отсутствие пути – аналогично точному методу, ожидается INF.
3. Треугольник – жадный алгоритм также находит оптимальный маршрут 17.
4. Неоптимальный случай – специально построенная матрица, на которой жадный алгоритм даёт стоимость 202, тогда как точный метод находит 104 (оптимум).

Все тесты успешно пройдены. Для точного метода при $n \leq 15$ время выполнения не превышает долей секунды. Жадный алгоритм работает практически мгновенно для любых n в рамках задания.

Вывод

В ходе выполнения лабораторной работы были реализованы два алгоритма для задачи коммивояжёра: точный метод – рекурсивное динамическое программирование с битовыми масками. Алгоритм гарантированно находит оптимальный маршрут, но имеет экспоненциальную сложность $O(n^2 \cdot 2^n)$, что ограничивает его применимость значениями $n \leq 15$ (согласно условию).

Приближённый метод АЛШ-1 – жадный алгоритм ближайшего соседа. Он работает за $O(n^2)$ и находит приемлемое решение, однако не гарантирует оптимальности. Для ряда матриц разница в стоимости может быть существенной. Программа поддерживает генерацию произвольных и симметричных матриц расстояний, сохранение их в файл и чтение из файла, что позволяет гибко тестировать алгоритмы на различных входных данных. Реализован подробный вывод промежуточных вычислений для отслеживания хода работы алгоритмов. Проведённое тестирование подтверждает корректность работы обоих алгоритмов в рамках поставленной задачи. Таким образом, цель работы достигнута: изучены и реализованы точный и приближённый методы решения задачи коммивояжёра, проведён анализ их вычислительной сложности.